Constructors in Java

27 May 2025 11:34

A constructor in Java is a block of codes like the method. It is called when an instance of a class is created. at a time of calling constructor memory for the object is allocated in the memory It is a special type of method which is used to initialize the object.

Every time when we create an object by using the new keyword, it calls a default constructor. If there is no constructor available in the class, in such case, Java compiler provides a default constructor by default.

```
RailwayTicket rt = new RailwayTicket();
```

[no-argument constructor].

Keyword. Java compiler is creating this constructor.

There are two types of constructor in Java: **No-argument constructor and parameterized constructor.**

```
Scanner sc = new Scanner(System.in);
```

Keyword Constructor Parameter

[Parameterized constructor]

The job of 'new' keyword is to allocated memory on the Heap.

Note: It is called constructor because it constructs the values at the time of object creation. It is not necessary to write a constructor of a class. because Java compiler creates a default constructor, if your class does not have any.

Rules for Creating Java Constructor

There are the following rules defined for defining a constructor:

1. Constructor name must be the same as its class name.
2. A constructor must have no explicit return type.
3. A Java constructor cannot be abstract, static, final and synchronized.

Note: We can use access modifiers while declaring a constructor. It controls the object creation. A constructor may be private, protected, public or default.

Types of Java Constructors:

There are two types of constructor in Java:

1. Default Constructor(No-arg constructor)
2. Parametrized constructor.

1. **Java Default Constructor:** When a constructor does not have any parameter, it is also known as default constructor.

Syntax:

```
      <class_name>(){
         //setup instance variables;
      }
```

For example:

```
package ClassAndObjects;


/**
 * Write a description of class Bike here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Bike
```

```
{
    // instance variables - replace the example below with your own
    private int x;
    private int chasisNum;

    /**
     * Constructor for objects of class Bike
     */
    public Bike() //Default constructor
    {
        // initialise instance variables
        x = 100;
        chasisNum = 9999999;
    }

    /**
     * An example of a method - replace this comment with your own
     *
     * @param  y  a sample parameter for a method
     * @return    the sum of x and y
     */
    public int sampleMethod(int y)
    {
        // put your code here
        return x + y;
    }
}
```

is use initialize instance variable.

int → 0
byte → 0
short → 0
long → 0L

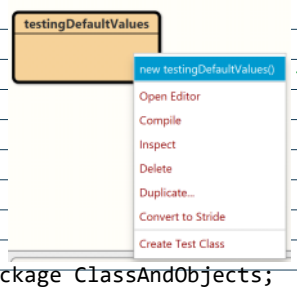
default value assigned by Java-compiler, when default-constructor is created by the compiler.

double → 0.0
float → 0.0f

boolean → false

char → '\0' → null.

String → null.

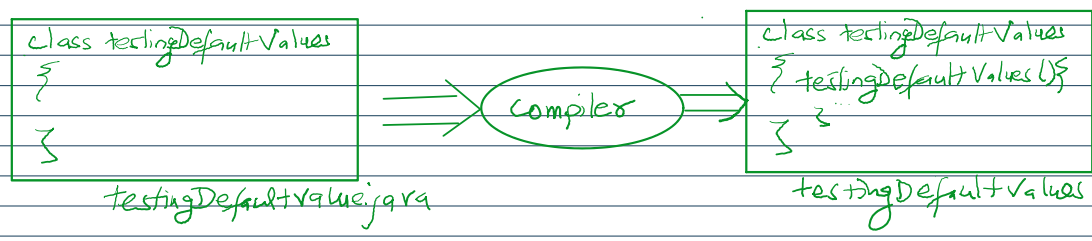


← constructor created by Java compiler.

```
package ClassAndObjects;


/**
 * Write a description of class testingDefaultValues here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class testingDefaultValues
{
    boolean b;
    byte bt;
    short s;
    int i;
    long l;
    float f;
    double d;
    char c;
    String str;

    public void display(){
        System.out.print("\fDefault values assigned by Java compiler"+
        "\nboolean = " + b +
        "\nbyte = " + bt+
        "\nshort = " + s +
        "\nint = " + i +
        "\nlong = " + l+
        "\nfloat = " + f +
        "\ndouble = " + d+
        "\nchar = " + c +
        "\nString = " + str);
    }
```



```
}
```

BlueJ: Terminal Window - AaditClass7

Options

```
Default values assigned by Java compiler
boolean = false
byte = 0
short = 0
int = 0
long = 0
float = 0.0
double = 0.0
char = 
String = null
```

→ What is the purpose of Default Constructor?

The default constructor is used to provide the default values to the object like '0', null etc. depending on the data type of instance variable.

Java Parameterized Constructor

A constructor that has a specific number of parameters is called a parameterized constructor.

Why use the parameterized constructor?

The parameter is constructor is used to provide different values to distinct objects. However, you can provide the same values also.

Example of parameterized constructor:

```
package ClassAndObjects;

/**
 * Write a description of class Bike here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Bike
{
    // instance variables - replace the example below with your own
    private int regNum;
    private int chasisNum;
    private String manufName;
    /**
     * Constructor for objects of class Bike
     */
    public Bike() //Default constructor
    {
        // initialise instance variables
        regNum = 0;
        chasisNum = 0;
    }

    /**
     * Parameterized constructor
     * @param  rn  a registration number
     * @param cn a chasis number
     * @param mn a manufacturer name
     */

    public Bike(int rn, int cn, String mn){
        regNum = rn;
        chasisNum = cn;
        manufName = mn;
    }
    /**
     * @return returns String format of the given object.
     */
    @Override
    public String toString(){
        return "[regNum = " + regNum + ", chasisNum = " + chasisNum +
        ", manufName = " + manufName + "]";
```

```
    }
}

package ClassAndObjects;
import java.util.Scanner;


/**
 * Write a description of class CreatingBikes here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class CreatingBikes
{
    public static void main(String args[]){
        Scanner sc = new Scanner(System.in);
        System.out.print("\f");
        int cn, rn;
        String mn;
        System.out.print("Enter registration number: ");
        rn = sc.nextInt();
        System.out.print("Enter chasis number: ");
        cn = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter manufacturer name: ");
        mn = sc.nextLine();
        Bike honda = new Bike(rn, cn, mn);
        System.out.print("\n" +  honda);
        Bike hero = new Bike(5254, 3456, "Hero");
        System.out.print("\n" + hero);
    }
}
```

Output:

```
BlueJ: Terminal Window - AaditClass7
Options
Enter registration number: 5123
Enter chasis number: 34567
Enter manufacturer name: Honda

[regNum = 5123, chasisNum = 34567, manufName = Honda]
[regNum = 5254, chasisNum = 3456, manufName = Hero]
```

Question 6

Given are two strings, input string and a mask string that remove all the characters of the mask string from the original string. **[10]**

Example: INPUT: ORIGINALSTRING: **communication**
MASK STRING: **mont**
OUTPUT: **cuicai**

A class **StringOp** is defined as follows to perform above operation.

Some of the members of the class are given below:

Class name	:	**StringOp**
Data members/instance variables:		
str	:	to store the original string
msk	:	to store the mask string
nstr	:	to store the resultant string
Methods / Member functions:		
StringOp()	:	default constructor to initialize the data member with legal initial value
void accept( )	:	to accept the original string str and the mask string msk in lower case
void form()	:	to form the new string nstr after removal of characters present in mask from the original string
void display( )	:	to display the original string and the newly formed string nstr

Specify the class **StringOp** giving details of the **constructor( )**, **void accept( )**, **void form()** and **void display( )**. Define a **main( )** function to create an object and call all the functions accordingly to enable the task. (Create)

Question 7 **[10]**

A class **Mixarray** contains an array of integer elements along with its capacity (More than or equal to 3). Using the following description, form a new array of integers which will contain only the first 3 elements of the two different arrays one after another.

Example: Array1: { 78, 90, 100, 45, 67 }
Array2: {10, 67, 200, 90 }
Resultant Array: { 78, 90, 100, 10, 67, 200}

The details of the members of the class are given below:

Class name	:	**Mixarray**
Data members/instance variables:		
arr[]	:	integer array
cap	:	integer to store the capacity of the array
Member functions/methods:		
Mixarray (int mm)	:	to initialize the capacity of the array cap=mm
void input( )	:	to accept the elements of the array
Mixarray mix(Mixarray P, Mixarray Q)	:	returns the resultant array having the first 3 elements of the array of objects P and Q
void display( )	:	to display the array with an appropriate message.

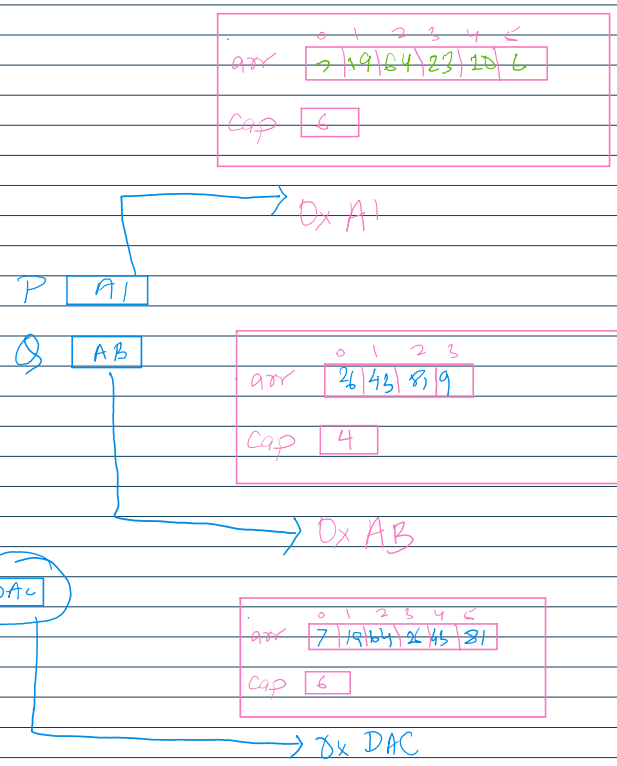
Specify the class **Mixarray**giving details of the **constructor(int)**, **void input( )**, **Mixarray mix(Mixarray,Mixarray)** and **void display( ).** Define a **main( )** function to create objects and call all the functions accordingly to enable the task. (Create)

```
import java.util.*;

/**

* Write a description of class Mixarray here.
```

```
*
* @author (your name)
* @version (a version number or a date)
*/
public class Mixarray
{
    private int arr[], cap;
    public Mixarray(int mm){
        cap = mm;
        arr = new int[cap];
    }
    public void input(){
        Scanner sc = new Scanner(System.in);
        int i;
        for(i=0;i<cap;i++){
            System.out.print("Enter a Value : ");
            arr[i] = sc.nextInt();
        }
    }
    public Mixarray mix(Mixarray P, Mixarray Q){
        Mixarray T = new Mixarray(6);
        int i, j;
        for(i=0, j=0;i<3;i++, j++){
            T.arr[j] = P.arr[i];
        }
        for(i=0;i<3;i++, j++){
            T.arr[j] = Q.arr[i];
        }
        return T;
    }
    public void display(){
        System.out.println();
        int i;
        System.out.print("[ ");
        for(i=0;i<cap-1;i++){
            System.out.print(arr[i] + ", ");
```



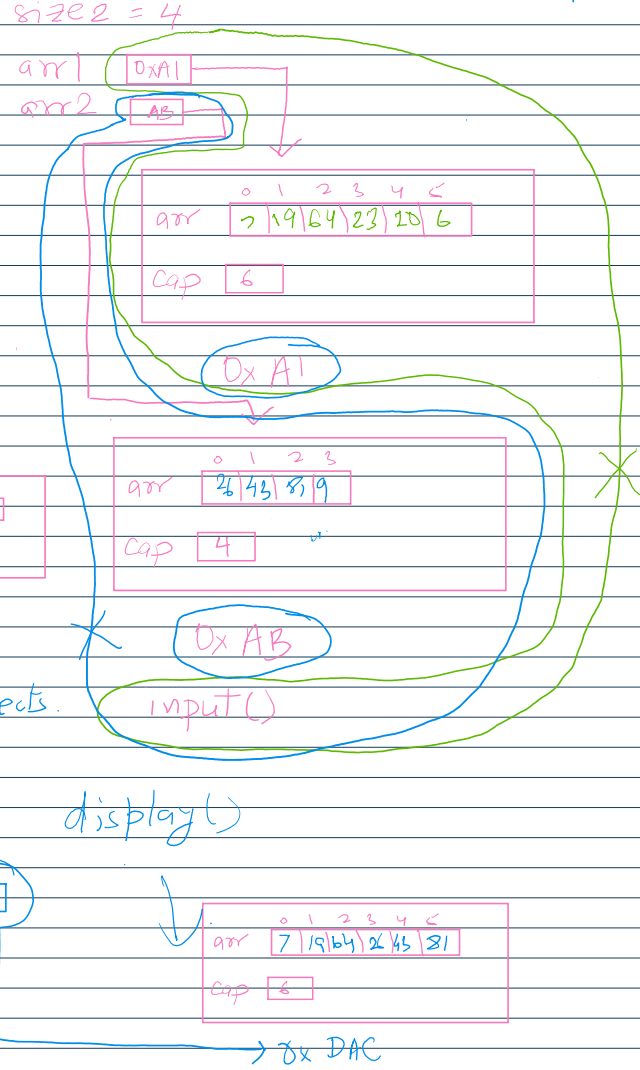
```
        }
        System.out.print(arr[i] + " ]\n");
    }
    public static void main(String args[]){
        System.out.print("\f");
        int size1, size2;
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Number of Elements in Array 1 : ");
        size1 = sc.nextInt();
        System.out.print("Enter Number of Elements in Array 2 : ");
        size2 = sc.nextInt();
        Mixarray arr1 = new Mixarray(size1);
        Mixarray arr2 = new Mixarray(size2);
        Mixarray result = new Mixarray(6);
        arr1.input();
        arr2.input();
        result=result.mix(arr1,arr2);
        System.out.print("\nArray 1 : ");
        arr1.display();
        System.out.print("\nArray 2 : ");
        arr2.display();
        System.out.print("\nResultant Array : ");
        result.display();
    }
}
```

created objects

reference variables

passing reference

function call

why? Because they keep the address of objects.

size1 = 6
size2 = 4